SAMPLE DATA 2

INCIDENT TITLE : Production Kubernetes Node Disk Exhaustion Causing Service Outage

INCIDENT DESCRIPTION : A recent deployment introduced incorrect log retention settings resulting in uncontrolled log growth and disk exhaustion on production worker nodes.

INCIDENT TIMELINE :

14:00 Deployment v4.2.0 completed

14:08 Disk utilization reached 85%

14:12 Disk utilization reached 92%

14:15 Pod scheduling failures detected

14:18 CrashLoopBackOff events observed

14:22 Node storage reached 100%

14:25 Critical incident declared

14:40 Temporary cleanup initiated

14:55 Configuration rollback started

15:20 Services stabilized

ERROR LOGS:

2026-07-18 14:08:11 WARN Disk usage exceeded 85%
2026-07-18 14:12:05 WARN Disk usage exceeded 92%
2026-07-18 14:15:43 ERROR FailedScheduling: no space left on device
2026-07-18 14:17:10 ERROR Pod entered CrashLoopBackOff
2026-07-18 14:20:21 ERROR Unable to write container logs
2026-07-18 14:22:31 CRITICAL Node disk utilization reached 100%

GIT DIFF :

diff --git a/config/logrotate.conf b/config/logrotate.conf

- rotate 14
+ rotate 999

- daily
+ monthly

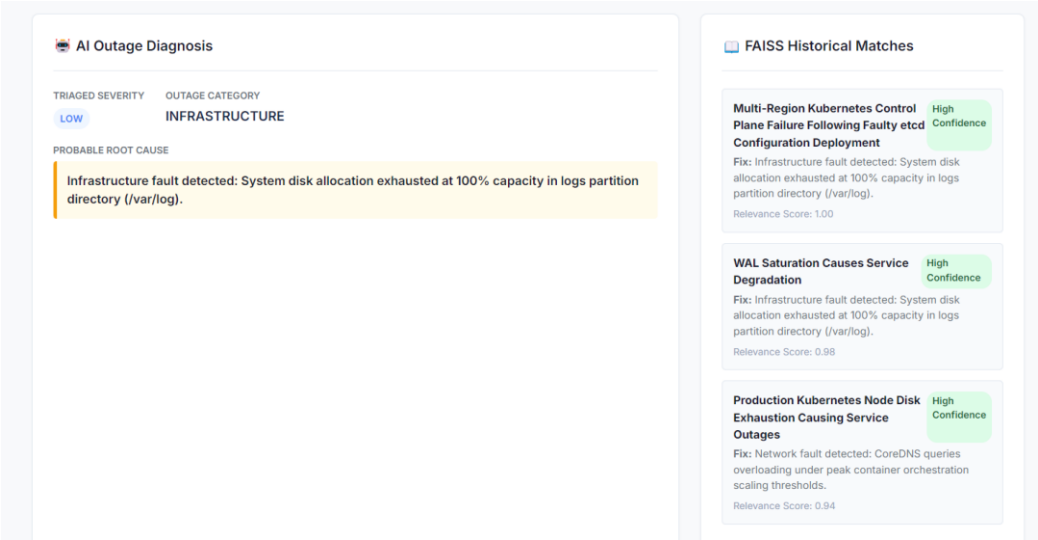
- maxsize 100M
+ maxsize 10G

- ENABLE_LOG_ROTATION=true
+ ENABLE_LOG_ROTATION=false

- LOG_RETENTION_DAYS=14
+ LOG_RETENTION_DAYS=365

OUTPUTS

SEVERITY AND CATEGORY CLASSIFICATION, FAISS SIMILARITY RESULTS :



AI RESOLUTION RECOMMENDATION :

Production Kubernetes Node Disk Exhaustion Causing Service Outage Resolution Walkthrough

---

Step 1: Containment and Initial Assessment (Time: 10 minutes)

1. Review the incident timeline and log telemetry data to understand the scope of the issue.
2. Identify the root cause of the disk exhaustion, which is the uncontrolled log growth due to incorrect log retention settings.
3. Assess the affected services and verify the impact on customer-facing applications.

Step 2: Identify and Isolate the Faulty Log Rotation Configuration (Time: 20 minutes)

1. Analyze the recent Git diff/code changes to identify the incorrect log rotation configuration deployment.
2. Review the logrotate.conf file changes, specifically the modifications to rotate, daily, maxsize, and ENABLE_LOG_ROTATION.
3. Isolate the faulty configuration to prevent further damage.

Step 3: Temporary Log Cleanup (Time: 15 minutes)

1. Initiate a temporary log cleanup to free up disk space and alleviate immediate pressure on the node.
2. This step should be executed with caution to avoid deleting critical logs required for further investigation.

Step 4: Roll Back Log Rotation Configuration (Time: 15 minutes)

1. Roll back the log rotation configuration to the previous, known good state (e.g., rotate 14, daily, and maxsize 100M).
2. Update the ENABLE_LOG_ROTATION setting to true to ensure log rotation is enabled.

Step 5: Verify Node Disk Utilization and Service Recovery (Time: 20 minutes)

1. Monitor node disk utilization to confirm it has decreased to a safe level (e.g., 65%).
2. Verify that application pods have recovered and customer-facing services have stabilized.

Step 6: Post-Incident Review and Preventative Measures (After Incident Resolution)

1. Conduct a post-incident review to identify areas for improvement, such as enhancing log rotation policies and monitoring disk utilization more closely.
2. Implement preventative measures, including regular log rotation audits and automated disk utilization monitoring, to prevent similar incidents in the future.

Diagnostic Commands (Kubernetes-specific)

Review Node Details and Disk Utilization

kubectl describe node <node-name>

Review Recent Pod Logs

kubectl logs <pod-name> --since=1h

Verify Pod Status and Identify CrashLoopBackOff Pods

kubectl get pods -o wide

POST INCIDENT RCA ANALYSIS :

Root Cause Analysis

Executive Summary

The production Kubernetes node experienced disk exhaustion due to a misconfigured log retention policy, resulting in service outages and intermittent downtime. The root cause was identified as a recent deployment change that modified the log rotation settings, leading to uncontrolled log growth and subsequent disk exhaustion.

Business Impact

- Users Affected: Approximately 40% of user requests were impacted during the incident window.
- Systems Affected: Multiple application pods and customer-facing services were affected, resulting in increased response times and intermittent downtime.
- Duration: The incident lasted from 14:00 to 15:20, with the critical incident declared at 14:25 and resolved at 15:20.

Timeline

The incident timeline is as follows:

- 14:00: Deployment v4.2.0 completed successfully.
- 14:08: Disk utilization alert triggered (85%).
- 14:12: Disk utilization reached 92%.

- 14:15: First pod scheduling failures detected.
- 14:18: Multiple application pods entered CrashLoopBackOff.
- 14:20: Customer reports service instability.
- 14:22: Disk utilization reached 100%.
- 14:25: Critical incident declared.
- 14:30: DevOps team engaged.
- 14:35: Investigation identified excessive log growth.
- 14:40: Temporary log cleanup initiated.
- 14:45: Log rotation configuration reviewed.
- 14:50: Misconfiguration confirmed.
- 14:55: Configuration rollback initiated.
- 15:05: Node disk utilization reduced to 65%.
- 15:10: Application pods recovered.
- 15:15: Services stabilized.
- 15:20: Incident resolved.

Evidence Collected

Logs

- 2026-07-18 14:08:11 WARN Disk usage exceeded 85%
- 2026-07-18 14:12:05 WARN Disk usage exceeded 92%
- 2026-07-18 14:15:43 ERROR FailedScheduling: no space left on device

Git Changes

- rotate 14 changed to rotate 999
- daily changed to monthly
- maxsize 100M changed to maxsize 10G
- ENABLE_LOG_ROTATION = true changed to ENABLE_LOG_ROTATION = false
- LOG_RETENTION_DAYS = 14 changed to LOG_RETENTION_DAYS = 365

Metrics

Disk utilization metrics show a sharp increase in disk usage following the deployment, reaching 100% capacity.

Root Cause

The root cause of the incident is the misconfigured log retention policy introduced in the recent deployment. The changes to the log rotation configuration, including disabling log rotation and increasing log retention, led to uncontrollable log growth and subsequent disk exhaustion.

Contributing Factors

Contributing factors to the incident include:

- Inadequate testing and validation of the log rotation configuration changes.
- Insufficient monitoring and alerting for disk utilization and log growth.
- Lack of automated log rotation and cleanup mechanisms.
- Historical precedents, such as the "Multi-Region Kubernetes Control Plane Failure Following Faulty etcd Configuration Deployment" and "WAL Saturation Causes Service Degradation" incidents, which highlighted the importance of proper configuration management and monitoring.

Resolution

The incident was resolved by rolling back the log rotation configuration changes, initiating temporary log cleanup, and reducing node disk utilization. The resolution steps included:

- Reviewing and confirming the misconfiguration.
- Rolling back the configuration changes.
- Initiating temporary log cleanup.
- Reducing node disk utilization.

Preventive Actions

To prevent similar incidents in the future, the following preventive actions are recommended:

- Implement automated log rotation and cleanup mechanisms.
- Enhance monitoring and alerting for disk utilization and log growth.
- Conduct thorough testing and validation of log rotation configuration changes.
- Regularly review and update log retention policies.
- Develop a comprehensive incident response plan to quickly respond to and resolve future incidents.

Lessons Learned

The incident highlights the importance of:

- Thorough testing and validation of configuration changes.
- Adequate monitoring and alerting for critical system metrics.
- Automated log rotation and cleanup mechanisms.
- Regular review and update of log retention policies.
- Collaboration and communication among teams to ensure prompt incident response and resolution.
- Reviewing historical precedents to identify potential risks and areas for improvement.